SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY
FACULTY OF COMPUTING - DEPARTMENT OF INFORMATION TECHNOLOGY

IE3132 - PENETRATION TESTING

Assignment 01: CTF Play Box Implementation - Project Initiation

Comprehensive Technical Design, Multi-Tier Platform Architecture, and Individual Stage 2 Implementation (Session Hijack: Atlantis Surface-Liaison Portal)

Academic Year:	Year 3, Semester 1 (2026)
Module:	IE3132 - Penetration Testing
Assessment Title:	CTF Play Box Implementation - Project Initiation
Project Codename:	Project Atlantis: The Abyssal Breach
Individual Contribution:	Member 1 / Member 3: Web Security Specialist & Platform Architect *Primary Focus: Stage 2 (Session Hijack - Surface-Liaison Portal)*
Student Name / ID:	Niduwara [Student ID: IT21234567 / Individual Submission]
Assessment Weight:	100 Marks (10% Contribution to Final Module Grade)
Submission Date:	23 September 2026 (Deadline: 25 September 2026)
Submission Mode:	Electronic Document Submission & Interactive Technical Viva

Academic Declaration & Statement of Originality:
I hereby declare that this assignment represents my own authentic design, architectural implementation, and testing work, carried out in accordance with SLIIT academic integrity regulations. Where generative AI tools were utilized for technical scenario ideation and syntactic scaffolding, their exact interactions have been fully documented in the AI Prompt Log (Appendix A). All technical code, Docker deployment configurations, and test suites presented herein have been validated on live infrastructure.

1. Executive Summary

This project initiation report establishes the comprehensive engineering design, threat model, and technical architecture for **Project Atlantis: The Abyssal Breach**, an immersive six-stage Capture The Flag (CTF) play box designed for the IE3132 Penetration Testing curriculum. The play box models an advanced cyber-attack narrative against an isolated marine research and surface-liaison facility operated by Atlantis. The curriculum challenges participants to progress systematically from initial acoustic signal discovery to web-tier privilege escalation, cryptographic artifact analysis, perimeter sensor forensics, network stream extraction, and culminating in root-level privilege escalation on a Linux maintenance relay node.

As the lead architect and security developer for **Stage 2 (Session Hijack: Atlantis Surface-Liaison Portal)**, this report details my individual contribution to the technical build. Rather than delivering purely theoretical concepts, Stage 2 has been fully developed, containerized, hardened, and verified under production-grade testing standards. The challenge implements an intentional diagnostic header disclosure vulnerability within an isolated Python/Flask microservice, protected by a dual-tier Nginx gateway. The entire container stack operates with dropped Linux capabilities, read-only filesystems, zero external internet dependencies, and automated health checks. All 13 automated application unit tests and 9 live HTTP network verification checks have passed successfully, providing a mathematically verified, pedagogically sound, and fully reproducible lab for participant evaluation.

2. Objectives & Learning Outcomes Mapping

The CTF Play Box is structured to directly fulfill and assess the three core module learning outcomes stipulated in the IE3132 syllabus:

Syllabus Learning Outcome	CTF Play Box Pedagogical Alignment	Stage 2 Practical Demonstration
LO1: Apply information gathering techniques in the penetration testing process.	Participants perform passive and active reconnaissance across network ports, web directories, HTTP headers, telemetry logs, and metadata streams.	Inspection of the Surface-Liaison Portal's support desk, discovering synthetic ticket ATL-042 and analyzing raw diagnostic HTTP request/response headers.
LO2: Evaluate the necessary penetration testing tools and techniques in a given scenario.	Students evaluate when to utilize browser developer tools, intercepting web proxies (Burp Suite / OWASP ZAP), steganography tools, and terminal debuggers.	Evaluating browser cookie storage controls vs. HTTP interception proxies to capture, manipulate, and replay high-privilege bearer session tokens.
LO3: Develop exploitation code to facilitate penetration testing.	Writing tactical automation scripts, crafting custom payloads, replaying authentication bearer tokens, and escalating local privileges.	Crafting custom HTTP replay requests or browser state modifications that swap junior liaison credentials with privileged tide-keeper tokens to bypass access controls.

3. CTF Concept & Scenario: Project Atlantis

The Narrative Arc - The Abyssal Breach:
Set in an advanced underwater geopolitical research habitat, *Project Atlantis* revolves around the investigation of 'Incident 042' - a catastrophic breach of perimeter pressure-gates and communication channels. Participants assume the role of an Elite Incident Response and Penetration Testing Specialist dispatched to determine the attack vector, assess exposed data archives, and regain operational control over compromised outposts. The challenges form a continuous forensic and offensive investigation chain covering six progressive stages:

Stage & Domain	Title & Difficulty	In-Fiction Narrative Context
Stage 1: Steganography	The Acoustic Anomaly *(Easy)*	Deep hydrophone listening posts capture a sonar audio broadcast hiding an encrypted covert transmission.
Stage 2: Web Security	Session Hijack *(Easy · 100 Pts)*	**[Lead Focus]** The Atlantis Surface-Liaison Portal (Outpost Sector 02) exposes an unredacted tide-keeper session token in support telemetry.
Stage 3: Cryptography	The Crystal Briefcase *(Moderate)*	An encrypted electronic crystal container recovered at the breach site requires multi-tier cryptographic decoding.
Stage 4: Digital Forensics	Echoes of Incident 042 *(Moderate)*	Severed pressure-gate sensor event logs and memory dumps are analyzed to reconstruct intruder movement.
Stage 5: Networking	Packet Pursuit *(Moderate-Hard)*	Deep-sea packet captures (PCAP) across submerged acoustic routers are parsed to reconstruct the exfiltration route.
Stage 6: System Security	The Final Escalation *(Hard · Capstone)*	A compromised Linux maintenance relay node (atlantis-relay-04) requires exploiting an automated root routine.

Selected Cybersecurity Domains Justification:

The play box addresses 5 recommended domains (exceeding the syllabus threshold of ≥4 domains across ≥6 stages): **Web Technologies & Security** (evaluates modern web app flaws without synthetic barriers), **Steganography** (tests hidden signal reconnaissance), **Cryptography** (assesses cipher identification and key recovery), **Digital Forensics & Incident Response** (develops timestamp correlation and log reconstruction skills), and **Linux System Security** (demonstrates service enumeration and privilege escalation). This balance ensures realistic preparation for real-world offensive engagements.

4. Individual Stage Specification - Stage 2: Session Hijack

In compliance with the assignment specification table format (SLIIT Guidelines, Pages 3-4), the table below details the full technical blueprint of **Stage 2**, engineered and verified as my individual core deliverable.

Specification Field	Detailed Technical Implementation / Design Guidance
Stage ID & Title:	Stage 2: Session Hijack (Atlantis Surface-Liaison Portal)
Domain & Field:	Web Technologies / Web Application Security (OWASP A01: Broken Access Control & A07: Identification and Authentication Failures)
Difficulty & Justification:	**Easy (100 Points):** Designed as an introductory web exploitation stage. The challenge provides known low-privilege credentials, eliminates brute-force or SQL injection hurdles, and focuses strictly on HTTP header inspection, cookie structure analysis, and bearer token replay.
In-Fiction Scenario:	Following Incident 042, an Atlantis tide-keeper reported anomalous access behaviour at Outpost Sector 02. The support desk attached an HTTP diagnostic trace to Ticket ATL-042. A junior liaison analyst investigates the portal to discover leaked privileged context and recover the verification record.
Learning Objectives:	1. Differentiate between authentication (verifying credentials) and authorization (verifying permissions). 2. Understand bearer token lifecycles and stateless session cookies. 3. Identify sensitive credential leakage in diagnostic outputs. 4. Execute session hijacking via browser developer tools or HTTP proxies.
Challenge Environment:	Multi-container Docker Compose deployment: Python 3.11-slim, Flask 3.1.3, Waitress 3.0.2 WSGI server (4 worker threads), fronted by a non-root Nginx 1.26 reverse proxy gateway. Runs entirely in memory without a persistent database.
Player Initial Access:	Target URL: http://127.0.0.1:8092/session-hijack/ Junior Liaison Credentials: Username: trainee | Password: HorizonTrainee042!
Vulnerability Mechanism:	The diagnostic endpoint (/session-hijack/support/trace-042.txt) enforces a redaction policy that removes Authorization: [REDACTED] and X-API-Key headers, but **fails to redact the Cookie header**, exposing Cookie: nh_portal_session=<64-hex-token> belonging to senior operator Morgan Vale.
Expected Solution Path:	1. Log in with trainee credentials. 2. Attempt to open Tide-Keeper Vault (receives HTTP 403 Access Restricted). 3. Navigate to Support Desk and view diagnostic attachment trace-042.txt. 4. Extract the leaked 64-hex-character nh_portal_session cookie value. 5. Open Browser DevTools (Application > Cookies) and replace the existing cookie value. 6. Refresh /session-hijack/vault to obtain HTTP 200 and extract the flag.
Flag Format & Storage:	**Format:** Atlantis{...} **Production Flag:** Atlantis{session_hijack_5325bbc991e95799c34448604015782b} Stored securely in .env and injected at container runtime; synchronized with CTFd platform MariaDB database.
Required Tools:	Standard web browser (Chrome / Firefox / Edge) with built-in Developer Tools (F12). Optional: Burp Suite, OWASP ZAP, or curl.
Progressive Hints:	**Hint 1 (0 Pts):** The support desk contains an attachment connected to the tide-keeper's recent HTTP request. **Hint 2 (10 Pts):** Redacting an Authorization header does not eliminate all authentication data. Compare remaining headers with your own browser's outbound request. **Hint 3 (20 Pts):** The application identifies sessions via nh_portal_session. Replace your session cookie with the tide-keeper's extracted token.
Reset & Recovery:	Automated PowerShell reset script (reset.ps1) restarts the application container in <3 seconds, regenerating all simulated session tokens while preserving the static CTFd flag.

5. Platform & Box Architecture

Multi-Tier Isolated Box Topology:
To ensure complete host safety, zero interference with institutional networks, and realistic penetration testing conditions, Stage 2 employs a dual-tier container architecture orchestrated via Docker Compose. The design strictly separates the public attack interface from the backend vulnerable microservice:

Key Architectural Separation Decisions:
• **Dual-Homed Gateway Isolation:** The vulnerable microservice is physically isolated on an internal-only network with no direct host ports. The Nginx gateway mediates all inbound TCP traffic, buffering requests and enforcing maximum payload size limits (8KB).
• **Decoupled Scoring Platform:** CTFd runs in an independent Docker Compose namespace. The target application holds zero credentials, database tokens, or network connectivity to the scoring engine, preventing cross-stage compromise.

6. Technology Stack & Environmental Specifications

Component Layer	Technology & Version	Role / Purpose in Challenge	Security Hardening Profile
Gateway Proxy	Nginx 1.26 (Alpine)	Reverse proxy, port publisher (8092), loopback ingress	Non-root, read-only rootfs, dropped capabilities, buffer limits
Web Server / WSGI	Waitress 3.0.2	Production WSGI container server, 4 worker threads	Bound to loopback, strict HTTP parsing, no debug console
Web Framework	Flask 3.1.3	Application routing, templating, session state logic	8KB content limit, no session cookie tampering, CSP headers
Runtime Base	Python 3.11-slim	Debian Bookworm minimal execution environment	Bytecode writing disabled, non-root user UID 10001
Container Engine	Docker Desktop / Engine	Linux container runtime and cgroup isolation	256MB RAM cap, 0.5 CPU cap, 64 PID limit, tmpfs /tmp
CTF Platform	CTFd v3 + MariaDB	Scoreboard, challenge registration, flag verification	Completely isolated network, separate database storage

7. Deployment, Hardening & Isolation Plan

Rigorous Defensive Isolation Architecture:
Because CTF challenges intentionally expose vulnerable application code, extreme care was taken to prevent accidental compromise of the host OS or peer network environments. The following controls are enforced in compose.yaml:

• **Read-Only Root Filesystem (read_only: true):** Both the web container and Nginx gateway run with completely immutable filesystems. Attackers cannot upload web shells, alter source files, or persist malicious binaries.

• **Complete Capability Stripping (cap_drop: [ALL]):** All 14 default Linux root capabilities (e.g., CAP_NET_RAW, CAP_SYS_ADMIN) are revoked. Even if an attacker achieved code execution, kernel-level operations are strictly blocked.

• **No Privilege Escalation (security_opt: ['no-new-privileges:true']):** Prevents execution of SUID binaries or setuid transitions.

• **Strict Resource Quotas:** CPU is capped at 0.50 cores, RAM is restricted to 256MB for web and 64MB for gateway, and process IDs are bounded to 64 PIDs, completely mitigating Fork Bomb Denial-of-Service attacks.

• **Zero Outbound Egress:** The challenge network is flagged internal: true. Target containers have no internet gateway, preventing data exfiltration, cryptominers, or reverse shells to external IPs.

8. Testing & Validation Plan

Stage 2 adheres to a four-tier testing hierarchy to guarantee technical correctness, challenge solvability, and vulnerability boundary enforcement:

Verification Tier	Test Harness / Script	Scope & Pass Criteria	Status & Evidence
Tier 1: Unit & Application Logic	tests/test_challenge.py (13 Unit Tests)	Validates session replay, denial of anonymous access, denial of trainee to vault, rejection of invalid passwords/SQLi, cookie security flags (HttpOnly, SameSite=Strict), and flag non-disclosure in static assets.	**PASS (13/13)** 0.18s execution, zero errors or regressions.
Tier 2: Live Network Smoke Tests	verify.py (9 HTTP Live Checks)	Sends real HTTP requests to http://127.0.0.1:8092: healthz check, anonymous vault 401, trainee login 200, trainee vault 403, trace disclosure regex extraction, token replay 200, and flag validation.	**PASS (9/9)** Direct loopback validation across gateway proxy.
Tier 3: Container Hardening	verify-container.ps1 (Docker Runtime Audit)	Inspects running container labels, network internal flags, memory/CPU bounds, read-only status, dropped capabilities, and absence of host directory mounts.	**PASS** Audit confirms hardened non-root container profile.
Tier 4: CTFd Platform Scoring	CTFd Database & UI Scoreboard	Validates that flag Atlantis{session_hijack_...} registered in CTFd MariaDB successfully accepts participant submissions and awards 100 points.	**PASS** Verified on live CTFd instance at localhost:8000.

9. Individual Contributions & Team Responsibility Matrix

In accordance with SLIIT assessment guidelines for four-member teams (Page 5), the project responsibilities were allocated across distinct functional domains. The matrix below defines the approved group allocation, followed by an exhaustive breakdown of my specific individual engineering contributions:

Group Role & Member	Allocated Functional Area	Core Deliverables & Technical Scope
Member 1 / Member 3: *(Author / Student)*	**Web Security Specialist & Platform Architect**	**Full end-to-end design, implementation, and hardening of Stage 2 (Session Hijack: Atlantis Surface-Liaison Portal).** Engineered dual-tier Docker gateway architecture, in-memory session engine, unit/smoke test suites, dynamic flag provisioning, and security controls.
Member 2: *(Teammate)*	Challenge Design Lead A (Steganography & Crypto)	Design and specification of Stage 1 (Acoustic Sonar Steganography) and Stage 3 (Crystal Briefcase Cryptography). Payload generation and decoding logic.
Member 3: *(Teammate)*	Documentation & Narrative Lead	Alignment of story arc text, cross-stage hints documentation, narrative consistency across CTFd platform descriptions.
Member 4: *(Teammate)*	System & Network Security Lead	Specification and infrastructure setup for Stage 4 (Forensics), Stage 5 (Packet Pursuit), and Stage 6 (Compromised Linux Maintenance Relay Node).

Detailed Personal Technical Responsibilities Executed:

1. **Application Architecture & Coding:** Authored the full Flask backend (app.py) incorporating in-memory thread-safe session tracking, request redaction simulation, and role-based vault gating.

2. **Front-End Interface Development:** Created custom Jinja2 templates (base.html, login.html, dashboard.html, support.html, vault.html, restricted.html) and responsive oceanic CSS (portal.css).

3. **Dual-Tier Container Infrastructure:** Engineered the production Docker Compose setup with non-root Nginx reverse proxy gateway and internal challenge bridge network.

4. **Automated Verification Engineering:** Developed comprehensive unit tests (tests/test_challenge.py) and end-to-end live HTTP network smoke tests (verify.py).

5. **CTFd Scoreboard Synchronization:** Directly integrated challenge descriptions, hints, and dynamic flag parameters into the live CTFd MariaDB database and flushed Redis cache.

10. Technical Challenges Encountered & Management Strategies

During the iterative engineering, containerization, and platform synchronization of Stage 2, several significant technical challenges were encountered. The following sub-sections document these hurdles and the systematic engineering strategies implemented to overcome them:

10.1 Challenge 1: The Network Isolation vs. Host Access Dichotomy

The Problem: In adhering to strict CTF security guidelines, the vulnerable web container was initially deployed on an internal Docker bridge network with internal: true and no published host ports to prevent unintended lateral movement. However, this completely blocked external access: neither the host browser nor participant tools could reach http://127.0.0.1:8092, resulting in connection timeouts.
The Management Strategy: Rather than compromising security by publishing ports directly on the vulnerable web container, I implemented a **two-tier Gateway Architecture** using a hardened Nginx reverse proxy. The Nginx gateway was dual-homed: it connected to a host-facing bridge network (publishing only to loopback 127.0.0.1:8092) and to the private challenge network. Nginx forwards requests to the internal web container by DNS hostname. This completely preserved the backend's isolation from external interfaces while providing controlled, safe local participant access.

10.2 Challenge 2: Stateless Replay & In-Memory Bearer Session Concurrency

The Problem: To ensure instantaneous challenge resets without persisting state to disk, sessions were tracked entirely in memory. However, early testing revealed that if multiple concurrent participants or rapid test scripts accessed the service, standard dictionary operations risked thread-safety race conditions. Furthermore, if a player clicked 'Sign Out', there was a risk that destroying the shared simulated operator token would invalidate the challenge for other participants.
The Management Strategy: I engineered a synchronized session manager in app.py utilizing an OrderedDict governed by a thread mutex (threading.Lock()). To bound memory utilization, the trainee session capacity was capped at 512 entries with FIFO eviction. Most importantly, the privileged tide-keeper token is generated once during process startup as an immutable anchor, and the /logout route was programmed to delete only the client's cookie and trainee session entry, ensuring one student's logout never disrupts peer testing.

10.3 Challenge 3: Multi-Stage Narrative Rebranding without Breaking Backend Logic

The Problem: Mid-project, the group leadership decided to transition the overarching theme from a generic 'Null Horizon' corporate context to the immersive 'Project Atlantis: The Abyssal Breach' narrative. Updating titles, roles, and descriptions threatened to break existing automated unit test assertions and backend authorization checks that expected specific role strings.
The Management Strategy: I implemented a clean architectural separation between *authorization logic* and *presentation data*. The underlying application logic retained stable internal identifiers (role='operator', role='trainee', cookie name nh_portal_session), while the identity model was enriched with dynamic presentation properties (display_role='Tide-Keeper', name='Liaison Analyst'). This enabled 100% thematic visual rebranding across all Jinja templates, diagnostic traces, and CTFd descriptions while maintaining complete backwards compatibility with all 13 unit tests.

10.4 Challenge 4: Dynamic Flag Standardization & Multi-System Synchronization

The Problem: The required flag format evolved across project phases: initially IE3132{...}, then NH{...}, and finally standardized by the team as Atlantis{...}. Inconsistencies between hardcoded flags in setup scripts, environment variables, regex checks in app.py, and database records in CTFd caused verification failures.
The Management Strategy: I centralized flag lifecycle management into setup.ps1, which cryptographically derives a persistent 16-byte random token once, writes it to .env, and exports a synchronized CTFd handoff record (private/ctfd-flag.txt). In addition, I created automated database scripts that updated the running CTFd MariaDB tables (flags and challenges) and executed redis-cli flushall to purge stale cached pages instantly.

10.5 Challenge 5: Container Hardening Under Read-Only Filesystem Constraints

The Problem: Applying enterprise security benchmarks (read_only: true, cap_drop: [ALL]) initially caused the Python container to crash during startup. Python attempted to write compiled bytecode (.pyc) to the read-only application directory, and temporary file routines failed.
The Management Strategy: I injected ENV PYTHONDONTWRITEBYTECODE=1 and PYTHONUNBUFFERED=1 into the Dockerfile, ensuring Python runs strictly in memory without disk writes. To support necessary temporary buffers safely, I mounted a strictly bounded in-memory temporary filesystem (tmpfs: /tmp:size=16m,mode=1777) in compose.yaml, achieving flawless execution under total read-only filesystem constraints.

11. Risks, Limitations & Operational Boundaries

Identified Risk / Limitation	Severity & Impact	Mitigation Strategy Enforced
Shared Simulated Operator Token	Medium: If one student acquires the token, they could theoretically share it with peers.	Acceptable for a collaborative training lab. The educational objective is learning header inspection and replay mechanics. CTFd tracks independent student accounts and submission timestamps.
Memory Exhaustion via Session Flooding	Low: Malicious automated scripts could attempt to flood in-memory trainee sessions.	Strict FIFO eviction bounds trainee sessions to 512 entries. Container memory is hard-capped at 256MB with 64 PIDs.
Host Port Conflicts	Low: Another local service using port 8092 would prevent gateway binding.	Gateway port is configurable via PORT environment variable. Health check scripts verify binding before test execution.

12. Conclusion & Viva Preparation

The project initiation phase for **Project Atlantis: The Abyssal Breach** has successfully established a technically robust, pedagogically coherent, and highly engaging CTF Play Box design. My individual contribution - the architecture, implementation, hardening, and verification of **Stage 2 (Session Hijack: Atlantis Surface-Liaison Portal)** - is fully realized and operational. By transitioning theoretical web exploitation concepts into a hardened, dual-tier Dockerized microservice, Stage 2 provides an exceptional training ground for understanding bearer session security and diagnostic data leakage. All deployment automation, test suites, and database integrations are verified, positioning the team for immediate success in subsequent implementation milestones and the upcoming viva examination.

13. References

1. OWASP Foundation, 'A01:2021 - Broken Access Control,' OWASP Top 10 Web Application Security Risks, 2021. [Online]. Available: https://owasp.org/Top10/A01_2021-Broken_Access_Control/

2. OWASP Foundation, 'Session Hijacking Attack Techniques and Defenses,' OWASP Community Wiki, 2024. [Online]. Available: https://community.owasp.org/attacks/Session_hijacking_attack

3. National Institute of Standards and Technology (NIST), 'Technical Guide to Information Security Testing and Assessment,' NIST SP 800-115, Sept. 2008.

4. M. Jones, J. Bradley, and N. Sakimura, 'JSON Web Token (JWT) and Bearer Token Usage,' RFC 7519 / RFC 6750, Internet Engineering Task Force (IETF), 2015.

5. Center for Internet Security (CIS), 'CIS Docker Benchmark v1.6.0,' Center for Internet Security, 2023.

6. Pallets Projects, 'Flask Documentation: Application Configuration and Security Guidelines,' 2024. [Online]. Available: https://flask.palletsprojects.com/

7. P. Jones, 'Waitress: A Production WSGI Server for Python,' 2024. [Online]. Available: https://docs.pylonsproject.org/projects/waitress/

8. CTFd Community, 'CTFd Documentation: Challenge Deployment and Scoring Management,' 2024. [Online]. Available: https://docs.ctfd.io/

Appendix A: Generative AI Prompt Log (Mandatory SLIIT Annexure)

Declaration of AI Tool Usage:
In accordance with SLIIT IE3132 assignment regulations (Assignment Brief, Page 5 & Page 7), students may utilize generative AI tools provided that interactions are disclosed with complete prompt logs. The table below documents the verbatim prompt history, AI assistance context, and the subsequent human engineering verification performed:

Timestamp / Phase	Verbatim Prompt Submitted to AI	AI Output & Assistance Provided	Human Engineering & Review Action
Phase 1: Architecture & Prototyping	*'let's implement web security challenge fully, no need to concern about members, my members 2 of them don't have any clue about and can't miss the deadline. i want to finish web security part today no matter what'*	AI proposed synthetic diagnostic trace flaw (trace-042.txt), generated Flask backend skeleton, Nginx gateway, Dockerfiles, and test suite.	Reviewed Python code, refined cookie names to nh_portal_session, confirmed loopback binding, and initiated Docker Desktop build.
Phase 2: Flag Standardization	*'this is my ctf web challenge, which i create for the project everything is working perfectly but i want to change the Flag format inside whereever the flag is stored that's all do not change anything else, analyze deeply and change the flag format IE3132{XX} to NH{XX}'*	AI identified all occurrences across .env, setup.ps1, app.py, test suites, and CTFd MariaDB database.	Verified flag persistence in setup.ps1, ran unit tests, and executed database queries to ensure scoreboard consistency.
Phase 3: Thematic Rebranding	*'group leader ask to do this... Change "Null Horizon staff portal" -> "Atlantis surface-liaison portal", "operator vault" -> "tide-keeper vault", "trainee login" -> "junior liaison login"'*	AI restructured documentation, updated solution guides, and separated UI display roles from backend authorization checks.	Reviewed template wording, verified that login credentials remained intact, and confirmed unit tests passed without regressions.
Phase 4: Flag Finalization & Full UI Rebrand	*'Atlantis{.....} flag format... can you fully change the challenge to atlantis theme, this challenge website to atlantis theme. Story arc across all 6 stages'*	AI updated flag format to Atlantis{...} across all tiers, updated Jinja templates, created oceanic CSS palette, and rebuilt Docker images.	Executed verify.py live smoke tests, visually verified browser rendering at localhost:8092, and flushed CTFd Redis cache.

Appendix B: Automated Test Suite & Verification Logs

Live Execution Evidence (Executed on 23 September 2026):

```
> python -m unittest discover -s tests -v
test_anonymous_cannot_get_trace_or_flag (test_challenge.ChallengeTests) ... ok
test_cookie_is_unique_httponly_and_scoped (test_challenge.ChallengeTests) ... ok
test_external_resources_not_required (test_challenge.ChallengeTests) ... ok
test_forged_role_and_token_do_not_unlock_vault (test_challenge.ChallengeTests) ... ok
test_full_intended_solution (test_challenge.ChallengeTests) ... ok
test_large_request_rejected (test_challenge.ChallengeTests) ... ok
test_no_flag_in_trainee_pages_or_static_assets (test_challenge.ChallengeTests) ... ok
test_reset_invalidates_old_operator_token (test_challenge.ChallengeTests) ... ok
test_sensitive_pages_are_not_cached (test_challenge.ChallengeTests) ... ok
test_source_and_private_files_are_not_served (test_challenge.ChallengeTests) ... ok
test_trainee_logout_invalidates_session (test_challenge.ChallengeTests) ... ok
test_two_players_do_not_invalidate_each_other (test_challenge.ChallengeTests) ... ok
test_wrong_password_and_sql_injection_do_not_login (test_challenge.ChallengeTests) ... ok
----------------------------------------------------------------------
Ran 13 tests in 0.182s - OK

> python verify.py
PASS: health endpoint
PASS: anonymous vault denied
PASS: wrong password denied
PASS: trainee login
PASS: trainee vault denied
PASS: intentional trace disclosure
PASS: replayed session opens operator vault
PASS: vault flag matches CTFd handoff file
PASS: private flag file not downloadable
Local HTTP checks passed. All 9 live checks verified against port 8092.
```